평가 및 벤치마킹 시스템 분석

1. 평가 시스템 개요

Open R1의 평가 및 벤치마킹 시스템은 LightEval 프레임워크를 기반으로 구축되어 있으며, 수학, 코딩, 과학 추론 등 다양한 영역에서 모델의 성능을 종합적으로 평가합니다.

1.1 시스템 아키텍처

```
모델 → vLLM 서버 → LightEval → 벤치마크 태스크 → 결과 수집 → HuggingFace Hub
```

1.2 핵심 구성 요소

- **LightEval**: 표준화된 평가 프레임워크
- **vLLM**: 고성능 추론 엔진
- **Slurm**: 클러스터 작업 관리
- **HuggingFace Hub**: 결과 저장 및 리더보드

2. 지원 벤치마크 (`src/open_r1/utils/evaluation.py`)

2.1 벤치마크 등록 시스템

```
def register_lighteval_task(
configs: Dict[str, str],
eval_suite: str,
```

```
task_name: str,
task_list: str,
num_fewshot: int = 0,
):
"""LightEval 태스크 설정 등록"""
# LightEval 형식으로 태스크 목록 포맷
task_list = ",".join(
f"{eval_suite}|{task}|{num_fewshot}|0"
for task in task_list.split(",")
)
configs[task_name] = task_list
```

2.2 현재 지원 벤치마크

수학 벤치마크

```
# MATH-500: 고등학교/대학 수준 수학 문제
register_lighteval_task(LIGHTEVAL_TASKS, "lighteval", "math_500", "math_500", 0)

# AIME 2024: 미국 수학 올림피아드 예선
register_lighteval_task(LIGHTEVAL_TASKS, "lighteval", "aime24", "aime24", 0)

# AIME 2025: 최신 수학 올림피아드 문제
```

```
register_lighteval_task(LIGHTEVAL_TASKS, "lighteval", "aime25", "aime25", 0)
```

과학 벤치마크

```
# GPQA Diamond: 대학원 수준 물리, 화학, 생물학 문제
register_lighteval_task(LIGHTEVAL_TASKS, "lighteval", "gpqa", "gpqa:diamond", 0)
```

코딩 벤치마크

```
# LiveCodeBench: 실시간 코딩 문제
register_lighteval_task(LIGHTEVAL_TASKS, "extended", "lcb", "lcb:codegeneration"


# LiveCodeBench v4: 최신 버전
register_lighteval_task(LIGHTEVAL_TASKS, "extended", "lcb_v4", "lcb:codegenerati
```

2.3 벤치마크 특성

| 벤치마크 | 영역 | 문제 수 | 난이도 | 평가 방식 |

|---------|------|---------|--------|----------|

| **MATH-500** | 수학 | 500 | 고등학교~대학 | 정확도 |

| **AIME 2024** | 수학 | 30 | 올림피아드 | 정확도 |

| **AIME 2025** | 수학 | 30 | 올림피아드 | 정확도 |

| **GPQA Diamond** | 과학 | 198 | 대학원 | 정확도 |

| **LiveCodeBench** | 코딩 | 400+ | 경쟁 프로그래밍 | 실행 결과 |

3. 평가 실행 시스템

3.1 벤치마크 실행 함수

```
def  run_lighteval_job(

benchmark: str,

training_args: Union["SFTConfig", "GRPOConfig"],

model_args: "ModelConfig",

) -> None:

"""LightEval 작업 실행"""

# 1. 태스크 설정 가져오기

task_list = LIGHTEVAL_TASKS[benchmark]

model_name = training_args.hub_model_id

model_revision = training_args.hub_model_revision

# 2. GPU 수 결정

num_gpus = get_gpu_count_for_vllm(model_name, model_revision)

# 3. 텐서 병렬화 여부 결정

if get_param_count_from_repo_id(model_name) >= 30_000_000_000:

tensor_parallel = True # 30B+ 모델은 텐서 병렬화 필요

else:

num_gpus = 2  # 작은 모델은 2 GPU 사용

tensor_parallel = False

# 4. Slurm 명령 구성
```

```
cmd = VLLM_SLURM_PREFIX.copy()

cmd_args = [

f"--gres=gpu:{num_gpus}",

f"--job-name=or1_{benchmark}_{model_name.split('/')[-1]}_{model_revision}",

"slurm/evaluate.slurm",

benchmark,

f'"{task_list}"',

model_name,

model_revision,

f"{tensor_parallel}",

f"{model_args.trust_remote_code}",

]

# 5. 시스템 프롬프트 처리 (선택적)

if training_args.system_prompt is not None:

# Base64 인코딩으로 특수 문자 문제 해결

prompt_encoded = base64.b64encode(

training_args.system_prompt.encode()

).decode()

cmd_args.append(prompt_encoded)

# 6. 명령 실행

cmd[-1] += " " + " ".join(cmd_args)

subprocess.run(cmd, check=True)
```

3.2 배치 벤치마크 실행

```
def run_benchmark_jobs(
training_args: Union["SFTConfig", "GRPOConfig"],
model_args: "ModelConfig"
) -> None:
"""여러 벤치마크를 배치로 실행"""
benchmarks = training_args.benchmarks
# "all" 지정 시 모든 벤치마크 실행
if len(benchmarks) == 1 and benchmarks[0] == "all":
benchmarks = get_lighteval_tasks()
# 각 벤치마크 순차 실행
for benchmark in benchmarks:
print(f"Launching benchmark `{benchmark}`")
if benchmark in get_lighteval_tasks():
run_lighteval_job(benchmark, training_args, model_args)
else:
raise ValueError(f"Unknown benchmark {benchmark}")
```

4. Slurm 평가 스크립트 (`slurm/evaluate.slurm`)

4.1 환경 설정

```
#!/bin/bash
#SBATCH --ntasks-per-node=1
#SBATCH --gres=gpu:8
#SBATCH --partition=hopper-prod
#SBATCH --output=./logs/%x-%j.out
#SBATCH --error=./logs/%x-%j.err
#SBATCH --requeue
#SBATCH --time=1-00:00:00


# HuggingFace 클러스터 최적화 설정
module load cuda/12.4


# Weka 파일시스템 캐시 새로고침
echo "Refreshing Weka filesystem..."
find -L /fsx/h4/ -type f | xargs -d '\n' -r -n512 -P64 weka fs tier


# vLLM 멀티프로세싱 설정
export VLLM_WORKER_MULTIPROC_METHOD=spawn
```

4.2 매개변수 처리

```
# 명령행 인수 파싱
```

```
TASK_NAME=$1  # 벤치마크 이름
TASKS=$2  # LightEval 태스크 목록
MODEL_ID=$3  # 모델 ID
MODEL_REVISION=$4  # 모델 리비전
TENSOR_PARALLEL=$5  # 텐서 병렬화 여부 (선택적)
TRUST_REMOTE_CODE=$6  # 원격 코드 신뢰 여부 (선택적)
# $7은 시스템 프롬프트용 예약


# GPU 수 자동 감지
NUM_GPUS=$(nvidia-smi  -L | wc  -l)
```

4.3 모델 설정

```
# 텐서 병렬화 사용 시
if [ "$TENSOR_PARALLEL" = "True" ];  then
MODEL_ARGS="model_name=$MODEL_ID,revision=$MODEL_REVISION,trust_remote_code=$TRU
else
# 데이터 병렬화 사용
MODEL_ARGS="model_name=$MODEL_ID,revision=$MODEL_REVISION,trust_remote_code=$TRU
fi
```

4.4 LightEval 실행

```
# 출력 디렉토리 설정
OUTPUT_DIR="eval_results/$MODEL_ID/$MODEL_REVISION/$TASK_NAME"

# DeepSpeed 환경 변수 비활성화 (평가 시 충돌 방지)
ACCELERATE_USE_DEEPSPEED=false

echo "Running lighteval script ..."
echo "Eval results will be saved to $OUTPUT_DIR"

# LightEval 실행
lighteval vllm "$MODEL_ARGS" $TASKS \
--use-chat-template \
--output-dir $OUTPUT_DIR \
--save-details \
${7:+--system-prompt "$(echo "$7" | base64 --decode)"}
```

4.5 결과 업로드

```
# 리더보드 업로드
LM_EVAL_REPO_ID="open-r1/open-r1-eval-leaderboard"
OUTPUT_FILEPATHS=$(find $OUTPUT_DIR/results/ -type f \( -name "*.json" \))

for filepath in $OUTPUT_FILEPATHS; do
```

```
echo "Uploading $filepath to Hugging Face Hub..."
filename=$(basename -- "$filepath")
# 재시도 로직 (최대 20회)
for attempt in {1..20}; do
if huggingface-cli upload --repo-type space --private $LM_EVAL_REPO_ID $f
echo "Upload succeeded for $filepath"
break
else
echo "Upload failed for $filepath. Attempt $attempt of 20. Retrying in 5 second
sleep 5
fi
done
done

# 상세 결과 업로드
MODEL_NAME=$(echo $MODEL_ID | sed 's/\//_/g')
DETAILS_REPO_ID="open-r1/details-$MODEL_NAME"
DETAILS_FILEPATHS=$(find $OUTPUT_DIR/details/ -type f \( -name "*.parquet" \
TIMESTAMP=$(date +"%Y-%m-%dT%H-%M-%S")

python scripts/upload_details.py \
--data_files $DETAILS_FILEPATHS \
--hub_repo_id $DETAILS_REPO_ID \
```

```
--config_name $MODEL_REVISION.$TASK_NAME.$TIMESTAMP
```

5. 벤치마크 실행 스크립트 (`scripts/run_benchmarks.py`)

5.1 스크립트 인수

```
@dataclass
class ScriptArguments:
model_id: str = field(
default="deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B",
metadata={"help": "평가할 모델 ID"},
)
model_revision: str = field(
default="main",
metadata={"help": "모델 브랜치/리비전"}
)
trust_remote_code: bool = field(
default=False,
metadata={"help": "원격 코드 신뢰 여부"}
)
benchmarks: List[str] = field(
default_factory=lambda: [],
metadata={"help": "실행할 벤치마크 목록"}
```

```
)
list_benchmarks: bool = field(
default=False,
metadata={"help": "지원 벤치마크 목록 출력"}
)
system_prompt: Optional[str] = field(
default=None,
metadata={"help": "벤치마크용 시스템 프롬프트"}
)
```

5.2 메인 실행 함수

```
def  main():
parser = TrlParser(ScriptArguments)
args = parser.parse_args_and_config()[0]
# 지원 벤치마크 목록 출력
if args.list_benchmarks:
print("Supported benchmarks:")
for benchmark in SUPPORTED_BENCHMARKS:
print(f" - {benchmark}")
return
# 벤치마크 설정 구성
benchmark_args = SFTConfig(
```

```
output_dir="",
hub_model_id=args.model_id,
hub_model_revision=args.model_revision,
benchmarks=args.benchmarks,
system_prompt=args.system_prompt,
)
# 벤치마크 실행
run_benchmark_jobs(
benchmark_args,
ModelConfig(
model_name_or_path="",
model_revision="",
trust_remote_code=args.trust_remote_code
),
)
```

6. 실제 사용 예시

6.1 단일 벤치마크 실행

```
# MATH-500 벤치마크 실행
python  scripts/run_benchmarks.py \
--model_id open-r1/OpenR1-Distill-7B \
```

```
--model_revision  main \
--benchmarks math_500


# AIME 2024 벤치마크 실행
python  scripts/run_benchmarks.py \
--model_id open-r1/OpenR1-Distill-7B \
--benchmarks  aime24
```

6.2 다중 벤치마크 실행

```
# 여러 벤치마크 동시 실행
python  scripts/run_benchmarks.py \
--model_id open-r1/OpenR1-Distill-7B \
--benchmarks  math_500  aime24  gpqa


# 모든 벤치마크 실행
python  scripts/run_benchmarks.py \
--model_id open-r1/OpenR1-Distill-7B \
--benchmarks  all
```

6.3 시스템 프롬프트 사용

```
# 커스텀 시스템 프롬프트로 평가
```

```
python  scripts/run_benchmarks.py \
--model_id open-r1/OpenR1-Distill-7B \
--benchmarks  math_500 \
--system_prompt "You are a helpful math tutor. Solve problems step by step."
```

6.4 훈련 후 자동 평가

```
# YAML 설정에서 자동 평가 설정
benchmarks:
- math_500
- aime24
- gpqa

# 또는 모든 벤치마크
benchmarks:
- all
```

7. 성능 최적화

7.1 GPU 메모리 최적화

```
# 대형 모델 (30B+)용 설정
MODEL_ARGS="...,tensor_parallel_size=8,gpu_memory_utilization=0.8,..."
```

```
# 중형 모델용 설정
MODEL_ARGS="...,data_parallel_size=4,gpu_memory_utilization=0.9,..."

# 소형 모델용 설정
MODEL_ARGS="...,data_parallel_size=2,gpu_memory_utilization=0.95,..."
```

7.2 생성 매개변수 튜닝

```
# 수학 문제용 설정 (정확성 중시)
generation_parameters="{max_new_tokens:32768,temperature:0.1,top_p:0.95}"

# 코딩 문제용 설정 (다양성과 정확성 균형)
generation_parameters="{max_new_tokens:16384,temperature:0.6,top_p:0.9}"

# 일반 추론용 설정
generation_parameters="{max_new_tokens:8192,temperature:0.7,top_p:0.95}"
```

7.3 배치 크기 최적화

```
# GPU 메모리에 따른 배치 크기 조정
def  get_optimal_batch_size(gpu_memory_gb, model_size_b):
if model_size_b >= 30: # 30B+ 모델
```

```
return 1 if gpu_memory_gb < 80 else 2
elif model_size_b >= 7: # 7B-30B 모델
return 4 if gpu_memory_gb < 40 else 8
else: # 7B 미만 모델
return 8 if gpu_memory_gb < 24 else 16
```

8. 결과 분석 및 리더보드

8.1 결과 형식

JSON 결과 파일


```
{
"results": {
"math_500": {
"accuracy": 0.892,
"num_correct": 446,
"num_total": 500,
"stderr": 0.014
}
},
"config": {
"model": "open-r1/OpenR1-Distill-7B",
"revision": "main",
```

```
"temperature": 0.6,
"max_new_tokens": 32768
},
"git_hash": "abc123def456",
"date": "2025-01-15T10:30:00"
}
```

Parquet 상세 결과

```
# 각 문제별 상세 결과
{
"problem_id": "math_500_001",
"problem": "원본 문제 텍스트",
"gold_answer": "정답",
"model_output": "모델 생성 결과",
"extracted_answer": "추출된 답안",
"is_correct": True,
"reasoning_steps": 5,
"generation_time": 2.3
}
```

8.2 리더보드 시스템

HuggingFace Space 리더보드

```
# 리더보드 업데이트 스크립트

def update_leaderboard(results_path, model_name, benchmark):

# 결과 파일 로드

with open(results_path) as f:

results = json.load(f)

# 리더보드 데이터 업데이트

leaderboard_entry = {

"model": model_name,

"benchmark": benchmark,

"score": results["results"][benchmark]["accuracy"],

"date": results["date"],

"config": results["config"]

}

# HuggingFace Hub에 업로드

upload_to_hub(leaderboard_entry, "open-r1/open-r1-eval-leaderboard")
```

8.3 성능 비교

OpenR1-Distill-7B vs DeepSeek-R1-Distill-Qwen-7B

| 벤치마크 | OpenR1-Distill-7B | DeepSeek-R1-Distill-Qwen-7B | 차이 |

|---------|------------------|---------------------------|------|

| **AIME 2024** | 52.7 | 51.3 | +1.4 |

| **MATH-500** | 89.0 | 93.5 | -4.5 |

| **GPQA Diamond** | 52.8 | 52.4 | +0.4 |

| **LiveCodeBench v5** | 39.4 | 37.4 | +2.0 |

9. 커스텀 벤치마크 추가

9.1 새 벤치마크 등록

```
# 새 벤치마크 추가
register_lighteval_task(
LIGHTEVAL_TASKS,
"extended", # 커스텀 태스크는 "extended" 사용
"my_custom_benchmark",
"my_custom_task",
0  # few-shot 수
)

# 지원 벤치마크 목록에 추가
SUPPORTED_BENCHMARKS.append("my_custom_benchmark")
```

9.2 커스텀 평가 스크립트

```
# scripts/evaluation/extended_lighteval_tasks/my_custom_task.py
from lighteval.tasks.lighteval_task import LightevalTaskConfig
from lighteval.metrics import Metrics
```

```
def  my_custom_metric(predictions, references):
"""커스텀 평가 메트릭"""
correct = 0
total = len(predictions)
for pred, ref in  zip(predictions, references):
if evaluate_custom_logic(pred, ref):
correct += 1
return correct / total


# 태스크 설정
my_custom_task = LightevalTaskConfig(
name="my_custom_task",
prompt_function=my_prompt_function,
hf_repo="my-org/my-dataset",
hf_subset="default",
metric=[Metrics.custom_metric(my_custom_metric)],
hf_avail_splits=["test"],
evaluation_splits=["test"],
few_shots_split="train",
few_shots_select="sequential",
suite=["extended"],
generation_size=1024,
```

```
stop_sequence=["\n"],
output_regex=r"Answer: (.*)",
frozen=False,
)
```

9.3 평가 파이프라인 통합

```
# 커스텀 벤치마크 실행
python scripts/run_benchmarks.py \
--model_id your-model \
--benchmarks my_custom_benchmark
```

10. 모니터링 및 디버깅

10.1 평가 로그 모니터링

```
# Slurm 작업 상태 확인
squeue -u $USER

# 로그 파일 실시간 모니터링
tail -f logs/or1_math_500_OpenR1-Distill-7B_main-*.out

# 오류 로그 확인
```

```
tail -f logs/or1_math_500_OpenR1-Distill-7B_main-*.err
```

10.2 성능 프로파일링

```
import time
import psutil
from contextlib import contextmanager


@contextmanager
def profile_evaluation(benchmark_name):
"""평가 성능 프로파일링"""
start_time = time.time()
start_memory = psutil.virtual_memory().used
print(f"Starting evaluation: {benchmark_name}")
try:
yield
finally:
end_time = time.time()
end_memory = psutil.virtual_memory().used
duration = end_time - start_time
memory_used = (end_memory - start_memory) / 1024**3  # GB
print(f"Evaluation completed: {benchmark_name}")
print(f"Duration: {duration:.2f} seconds")
```

```
print(f"Memory used: {memory_used:.2f} GB")
```

10.3 오류 처리 및 복구

```
def  robust_evaluation(model_id, benchmark, max_retries=3):
"""견고한 평가 실행"""
for attempt in  range(max_retries):
try:
run_lighteval_job(benchmark, training_args, model_args)
print(f"Evaluation succeeded on attempt {attempt + 1}")
break
except Exception as e:
print(f"Evaluation failed on attempt {attempt + 1}: {e}")
if attempt == max_retries - 1:
print(f"All {max_retries} attempts failed for {benchmark}")
raise
else:
print(f"Retrying in 60 seconds...")
time.sleep(60)
```

평가 및 벤치마킹 시스템은 Open R1 모델의 성능을 객관적으로 측정하고 다른 모델들과 비교할 수 있는 표준화된 프레임워크를 제공합니다. LightEval과 vLLM의 조합으로 효율적이고 확장 가능한 평가 파이프라인을 구축했습니다.